

AI-Powered JEE Test Generation SaaS

Complete Product Blueprint & Development Prompt Guide

This document contains the complete product architecture, execution roadmap, phase-wise development prompts, SaaS planning, technical structure, AI workflows, and monetization strategy for building a scalable AI-powered JEE examination platform. Core modules include: - Question Paper Generator - Student CBT System - Institute SaaS Dashboard - AI Question Extraction Engine - Analytics Engine - Doubt Solving Marketplace - PDF Generation System - Scalable Backend Infrastructure

Phase 0 — Master Architecture & Planning

- Define complete SaaS architecture and technical stack.
- Lock frontend, backend, database, storage, authentication, and deployment decisions.
- Define coding standards, modular architecture, and scalability strategy.
- Create ER diagrams and role-based system planning.
- Decide core entities: users, institutes, questions, tests, attempts, analytics.

Development Objective:

This phase should be implemented as a separate production-grade module with proper APIs, reusable architecture, validation, security, and scalability planning.

AI Prompting Strategy:

Use structured prompts containing: architecture, workflows, schema definitions, API expectations, security requirements, UI expectations, edge cases, and scalability rules.

Important Engineering Rules:

- Never generate the entire project in one prompt.
 - Build backend before frontend.
 - Lock database schema early.
 - Maintain modular services.
 - Use proper validation and error handling.
 - Build reusable components.
 - Use scalable indexing and caching strategies.
-

Phase 1 — Authentication & Core Database

- Build authentication using JWT and refresh tokens.
- Implement RBAC (Role-Based Access Control).
- Create PostgreSQL schema using Prisma ORM.
- Add audit logs, sessions, institute management, and secure password flows.
- Implement rate limiting and validation.

Development Objective:

This phase should be implemented as a separate production-grade module with proper APIs, reusable architecture, validation, security, and scalability planning.

AI Prompting Strategy:

Use structured prompts containing: architecture, workflows, schema definitions, API expectations, security requirements, UI expectations, edge cases, and scalability rules.

Important Engineering Rules:

- Never generate the entire project in one prompt.
 - Build backend before frontend.
 - Lock database schema early.
 - Maintain modular services.
 - Use proper validation and error handling.
 - Build reusable components.
 - Use scalable indexing and caching strategies.
-

Phase 2 — Question Bank Management

- Build structured question storage system.
- Support LaTeX, images, MCQ, numerical, multi-correct questions.
- Add chapter/topic tagging.
- Implement search filters and pagination.
- Create moderation workflow for admins.

Development Objective:

This phase should be implemented as a separate production-grade module with proper APIs, reusable architecture, validation, security, and scalability planning.

AI Prompting Strategy:

Use structured prompts containing: architecture, workflows, schema definitions, API expectations, security requirements, UI expectations, edge cases, and scalability rules.

Important Engineering Rules:

- Never generate the entire project in one prompt.
 - Build backend before frontend.
 - Lock database schema early.
 - Maintain modular services.
 - Use proper validation and error handling.
 - Build reusable components.
 - Use scalable indexing and caching strategies.
-

Phase 3 — Dynamic Test Generation Engine

- Generate chapterwise, subjectwise, and full syllabus tests.
- Implement difficulty balancing.
- Avoid repeated questions.

- Support custom patterns and section logic.
- Generate intelligent random papers.

Development Objective:

This phase should be implemented as a separate production-grade module with proper APIs, reusable architecture, validation, security, and scalability planning.

AI Prompting Strategy:

Use structured prompts containing: architecture, workflows, schema definitions, API expectations, security requirements, UI expectations, edge cases, and scalability rules.

Important Engineering Rules:

- Never generate the entire project in one prompt.
 - Build backend before frontend.
 - Lock database schema early.
 - Maintain modular services.
 - Use proper validation and error handling.
 - Build reusable components.
 - Use scalable indexing and caching strategies.
-

Phase 4 — PDF Generation Engine

- Generate professional PDFs using Puppeteer.
- Support institute branding and logos.
- Render LaTeX equations properly.
- Generate answer keys and solution documents.
- Implement QR verification.

Development Objective:

This phase should be implemented as a separate production-grade module with proper APIs, reusable architecture, validation, security, and scalability planning.

AI Prompting Strategy:

Use structured prompts containing: architecture, workflows, schema definitions, API expectations, security requirements, UI expectations, edge cases, and scalability rules.

Important Engineering Rules:

- Never generate the entire project in one prompt.
 - Build backend before frontend.
 - Lock database schema early.
 - Maintain modular services.
 - Use proper validation and error handling.
 - Build reusable components.
 - Use scalable indexing and caching strategies.
-

Phase 5 — Student CBT Examination System

- Create NTA-style interface.
- Add timer, question palette, save & next.

- Implement autosave and reconnect support.
- Track cheating signals and tab switching.
- Support high concurrency.

Development Objective:

This phase should be implemented as a separate production-grade module with proper APIs, reusable architecture, validation, security, and scalability planning.

AI Prompting Strategy:

Use structured prompts containing: architecture, workflows, schema definitions, API expectations, security requirements, UI expectations, edge cases, and scalability rules.

Important Engineering Rules:

- Never generate the entire project in one prompt.
 - Build backend before frontend.
 - Lock database schema early.
 - Maintain modular services.
 - Use proper validation and error handling.
 - Build reusable components.
 - Use scalable indexing and caching strategies.
-

Phase 6 — Analytics Engine

- Generate student analytics dashboards.
- Track weak chapters and accuracy.
- Generate percentile predictions.
- Create institute batch analysis tools.
- Add ranking and comparison systems.

Development Objective:

This phase should be implemented as a separate production-grade module with proper APIs, reusable architecture, validation, security, and scalability planning.

AI Prompting Strategy:

Use structured prompts containing: architecture, workflows, schema definitions, API expectations, security requirements, UI expectations, edge cases, and scalability rules.

Important Engineering Rules:

- Never generate the entire project in one prompt.
 - Build backend before frontend.
 - Lock database schema early.
 - Maintain modular services.
 - Use proper validation and error handling.
 - Build reusable components.
 - Use scalable indexing and caching strategies.
-

Phase 7 — Institute SaaS Features

- Institute dashboard management.

- Teacher and batch management.
- Subscription and billing system.
- Custom branding and white-labeling.
- Usage analytics and permissions.

Development Objective:

This phase should be implemented as a separate production-grade module with proper APIs, reusable architecture, validation, security, and scalability planning.

AI Prompting Strategy:

Use structured prompts containing: architecture, workflows, schema definitions, API expectations, security requirements, UI expectations, edge cases, and scalability rules.

Important Engineering Rules:

- Never generate the entire project in one prompt.
 - Build backend before frontend.
 - Lock database schema early.
 - Maintain modular services.
 - Use proper validation and error handling.
 - Build reusable components.
 - Use scalable indexing and caching strategies.
-

Phase 8 — AI Question Extraction Engine

- Extract questions from PDFs and images.
- Use OCR and Math OCR pipelines.
- Auto-detect chapters and difficulty.
- Convert extracted content into structured DB records.
- Reduce manual question entry costs.

Development Objective:

This phase should be implemented as a separate production-grade module with proper APIs, reusable architecture, validation, security, and scalability planning.

AI Prompting Strategy:

Use structured prompts containing: architecture, workflows, schema definitions, API expectations, security requirements, UI expectations, edge cases, and scalability rules.

Important Engineering Rules:

- Never generate the entire project in one prompt.
 - Build backend before frontend.
 - Lock database schema early.
 - Maintain modular services.
 - Use proper validation and error handling.
 - Build reusable components.
 - Use scalable indexing and caching strategies.
-

Phase 9 — Doubt Solving Marketplace

- Student uploads doubt image or text.
- AI attempts initial response.
- Escalate unresolved doubts to human solvers.
- Implement payment and commission system.
- Create solver rating and response metrics.

Development Objective:

This phase should be implemented as a separate production-grade module with proper APIs, reusable architecture, validation, security, and scalability planning.

AI Prompting Strategy:

Use structured prompts containing: architecture, workflows, schema definitions, API expectations, security requirements, UI expectations, edge cases, and scalability rules.

Important Engineering Rules:

- Never generate the entire project in one prompt.
- Build backend before frontend.
- Lock database schema early.
- Maintain modular services.
- Use proper validation and error handling.
- Build reusable components.
- Use scalable indexing and caching strategies.

Phase 10 — Production Scaling & DevOps

- Move infrastructure to AWS/GCP.
- Implement Redis caching and queues.
- Add monitoring, logging, backups, and CDN.
- Optimize DB indexing and scaling.
- Prepare for large-scale concurrent users.

Development Objective:

This phase should be implemented as a separate production-grade module with proper APIs, reusable architecture, validation, security, and scalability planning.

AI Prompting Strategy:

Use structured prompts containing: architecture, workflows, schema definitions, API expectations, security requirements, UI expectations, edge cases, and scalability rules.

Important Engineering Rules:

- Never generate the entire project in one prompt.
- Build backend before frontend.
- Lock database schema early.
- Maintain modular services.
- Use proper validation and error handling.
- Build reusable components.
- Use scalable indexing and caching strategies.

Recommended Tech Stack

Layer	Recommended Technology
Frontend	Next.js + TypeScript + Tailwind + shadcn/ui
Backend	NestJS + Node.js
Database	PostgreSQL + Prisma ORM
Caching	Redis
Search	Elasticsearch / PostgreSQL Full Text Search
Storage	AWS S3 / Cloudinary
Authentication	JWT + Refresh Tokens
PDF Engine	Puppeteer
AI OCR	Mathpix / Tesseract / Vision APIs
Deployment	AWS / GCP / Docker

Revenue Model & Monetization

1. Teacher SaaS Plans

- Free Plan: 2 papers/month
- Basic: ■199/month
- Pro: ■999/month
- Institute Plan: ■4999+/month

2. Student Subscription

- Premium mock tests
- Advanced analytics
- Rank prediction
- Unlimited practice

3. Doubt Marketplace

- Students pay per doubt
- Platform keeps commission
- Hybrid AI + Human solving model

4. White Label SaaS

- Custom domains
- Institute branding
- Dedicated dashboards

Final Strategic Advice

The biggest strength of this business is not the frontend UI — it is the structured question database, dynamic test generation engine, scalable architecture, and automation pipeline. Focus first on: 1. Structured question management 2. Dynamic paper generation 3. Reliable CBT infrastructure Avoid overbuilding in the beginning. Start with institute-focused SaaS validation. Scale slowly into AI extraction and doubt marketplace systems. This idea has strong startup potential because it combines: - SaaS - AI - Marketplace - Education Infrastructure - Analytics - Scalable recurring revenue